

EMSI — École Marocaine des Sciences de l'Ingénieur
Département Informatique · Mars 2026



CAHIER DES CHARGES TECHNIQUE

Projet de cours — Python & Développement Web

JobTech Solutions

Système de Gestion et de Suivi des Entretiens avec Intelligence Artificielle

Réalisé par

- › Diouri Mehdi
- › El Kharrazi Ibtihal
- › Assaadi Mohamed Nadir

Informations

École : EMSI — Rabat
Matière : Programmation Python
Date : Mars 2026
Version : 1.0

Dédicaces

Nous dédions ce travail à nos familles, pour leur soutien inconditionnel et leurs encouragements constants qui nous ont permis de persévérer tout au long de ce projet. À nos parents, qui ont cru en nous et nous ont donné les moyens de nous épanouir dans nos études d'ingénierie.

Nous dédions également ce travail à l'ensemble du corps professoral du département Informatique de l'EMSI, et tout particulièrement à notre professeure de Python, dont la pédagogie exigeante et bienveillante nous a transmis non seulement des compétences techniques solides, mais aussi une véritable passion pour la programmation et le développement logiciel.

À tous ceux qui, de près ou de loin, ont contribué à l'aboutissement de ce projet — merci.

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

En premier lieu, nous remercions chaleureusement notre professeure de Python pour nous avoir confié ce projet ambitieux et stimulant. Ses conseils précis, sa disponibilité et son suivi attentif ont été essentiels à l'avancement du travail. Sa manière d'enseigner Python — en alliant théorie rigoureuse et applications concrètes — nous a fourni les bases solides sur lesquelles nous avons pu construire cette solution.

Nous remercions également la direction de l'EMSI (École Marocaine des Sciences de l'Ingénieur) pour la qualité de la formation dispensée et les ressources informatiques mises à notre disposition. Les enseignements reçus en développement web, bases de données, algorithmique et intelligence artificielle ont tous trouvé une application directe dans ce projet.

Enfin, nous remercions nos familles pour leur patience et leur soutien tout au long des longues sessions de travail qu'a nécessité ce projet.

Résumé

Le présent document constitue le cahier des charges technique du projet JobTech Solutions, une application web développée dans le cadre du projet Python confié par notre professeure à l'EMSI. Ce projet vise à moderniser et automatiser la gestion des entretiens professionnels, qu'il s'agisse d'entretiens de recrutement ou d'entretiens annuels d'évaluation.

La solution repose sur une architecture Django MVT avec une base de données MySQL, enrichie de modules d'Intelligence Artificielle utilisant SpaCy, BERT et Scikit-learn pour l'analyse automatique des CV et le calcul d'un score de compatibilité candidat/offre. Le système intègre également Celery pour les tâches asynchrones, ReportLab pour la génération de rapports PDF, et Matplotlib pour les tableaux de bord analytiques.

Ce document couvre l'ensemble des spécifications fonctionnelles, la conception technique détaillée (architecture, diagrammes UML, schéma ERD), l'allocation des tâches entre les membres de l'équipe, et les résultats de réalisation.

Mots-clés : Django, Python, Intelligence Artificielle, NLP, Machine Learning, Gestion RH, MySQL, BERT, Scikit-learn, Celery.

Abstract

This document presents the technical specifications of the JobTech Solutions project, a web application developed as part of a Python programming course at EMSI. The project aims to modernize and automate the management of professional interviews, both for recruitment and annual performance reviews.

The solution is built on a Django MVT architecture with a MySQL database, enhanced by Artificial Intelligence modules using SpaCy, BERT, and Scikit-learn for automatic CV analysis and compatibility scoring. The system also integrates Celery for asynchronous tasks, ReportLab for PDF report generation, and Matplotlib for analytical dashboards.

Keywords: Django, Python, Artificial Intelligence, NLP, Machine Learning, HR Management, MySQL, BERT, Scikit-learn, Celery.

Table des Matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Abstract	iv
Liste des Figures	v
Liste des Abréviations	vi
Chapitre 1 : Contexte Général du Projet	1
Introduction	1
1. Présentation du Projet	1
1.1 Contexte et Problématique	1
1.2 Objectifs	2
1.3 Étude de l'Existant	2
2. Organisation	3
2.1 Équipe Projet	3
2.2 Planification	3
2.3 Budget	4
Chapitre 2 : Conception Technique	5
Introduction	5
1. Architecture Technique Détaillée	5
1.1 Vue Globale des Couches	5
1.2 Composants par Couche	6
2. Modèle de Classes UML	7
2.1 Classes et Méthodes	7
2.2 Relations et Cardinalités	8
3. Schéma ERD (Base de Données)	8
3.1 Modèle Entité-Relation	8
3.2 Contraintes et Index	9
4. Allocation des Tâches	9
4.1 Répartition par Membre	9
4.2 Tableau Détaillé	10
4.3 Planification par Sprint	11
Chapitre 3 : Analyse et Conception	12
Introduction	12
1. Besoins Fonctionnels	12
2. Architecture Globale	14
3. Stack Technique	15
4. Modélisation UML	16
5. Exigences Non Fonctionnelles	18
Chapitre 4 : Réalisation et Interfaces	19
1. Infrastructure et Déploiement	19
2. Interfaces de l'Application	20
3. Pipeline IA	21
Conclusion Générale	22
Références	23

Liste des Figures et Tableaux

- Figure 1 : Diagramme des cas d'utilisation — Vue par acteur
- Figure 2 : Diagramme de séquence — Pipeline IA : Upload et Analyse de CV
- Figure 3 : Architecture du système — Pattern MVT Django
- Figure 4 : Diagramme de classes — Modèle de données MySQL
- Figure 5 : Architecture technique détaillée — Composants par couche
- Figure 6 : Diagramme de classes UML complet avec méthodes
- Figure 7 : Schéma Entité-Relation (ERD) — Base de données MySQL
- Tableau 1 : Équipe projet et répartition des rôles
- Tableau 2 : Planification par sprint
- Tableau 3 : Budget global alloué au projet
- Tableau 4 : Allocation détaillée des tâches par module
- Tableau 5 : Stack technologique
- Tableau 6 : Exigences non fonctionnelles

Liste des Abréviations

Sigle	Définition
MVT	Model-View-Template : patron d'architecture Django (variante MVC)
NLP	Natural Language Processing : Traitement Automatique du Langage Naturel
ML	Machine Learning : Apprentissage Automatique
IA / AI	Intelligence Artificielle / Artificial Intelligence
ORM	Object-Relational Mapping : correspondance objet-relationnel Django
RBAC	Role-Based Access Control : contrôle d'accès par rôles
CRUD	Create, Read, Update, Delete : opérations de base sur les données
CSRF	Cross-Site Request Forgery : faille de sécurité web (protégée par Django)
XSS	Cross-Site Scripting : injection de scripts malveillants côté client
ERD	Entity-Relationship Diagram : schéma entité-relation base de données
CBV	Class-Based Views : vues Django orientées objet
WSGI	Web Server Gateway Interface : interface entre Python et serveur web
NER	Named Entity Recognition : reconnaissance d'entités nommées (SpaCy)
TF-IDF	Term Frequency-Inverse Document Frequency : méthode de vectorisation texte
PK / FK	Primary Key / Foreign Key : clé primaire / étrangère SQL
PDF	Portable Document Format : format de document portable (ReportLab)
API	Application Programming Interface : interface de programmation
AJAX	Asynchronous JavaScript and XML : requêtes HTTP asynchrones JS

Chapitre 1 : Contexte Général du Projet

Introduction

Dans le contexte des entreprises modernes, la gestion des ressources humaines est un levier stratégique majeur. Pourtant, les processus liés aux entretiens professionnels — recrutement, évaluation annuelle, suivi des performances — restent souvent fragmentés et peu outillés. Tableaux Excel, emails non structurés, documents Word éparpillés : ces pratiques engendrent des inefficacités coûteuses et des risques d'erreurs importants.

Le projet JobTech Solutions naît de ce constat. Conçu dans le cadre de notre cours Python à l'EMSI, il vise à développer une application web centralisée, automatisée et intelligente pour gérer l'intégralité du cycle de vie des entretiens professionnels. Ce premier chapitre pose le contexte du projet, analyse l'existant, définit les objectifs, et présente l'organisation de l'équipe.

1. Présentation du Projet

1.1 Contexte et Problématique

Les équipes RH font face à trois défis majeurs qui ont motivé ce projet :



Volume et vitesse : le nombre croissant de candidatures reçues par offre d'emploi rend la sélection manuelle non scalable et source de biais humains.



Fragmentation des outils : données dispersées entre Excel, emails, Outlook et documents Word, sans traçabilité ni cohérence.



Absence d'aide à la décision : les recruteurs manquent d'outils objectifs pour comparer les candidats selon des critères standardisés.

Ces problèmes engendrent des conséquences concrètes : délais de recrutement allongés, décisions subjectives, manque de traçabilité des évaluations, et impossibilité d'analyser les tendances RH dans le temps.

1.2 Objectifs du Projet

JobTech Solutions répond à ces problèmes avec cinq objectifs fondamentaux :

1. Centraliser toutes les données RH (offres, candidatures, entretiens, évaluations) dans une plateforme unique et accessible selon les rôles.
2. Automatiser les tâches répétitives : envoi de rappels, planification d'entretiens, génération de rapports PDF.
3. Intégrer l'Intelligence Artificielle pour analyser automatiquement les CV (NLP) et calculer un score de compatibilité objectif entre candidats et offres (ML).
4. Garantir sécurité et traçabilité : authentification robuste, RBAC à 4 niveaux, journal d'audit complet de toutes les actions CRUD.
5. Fournir des tableaux de bord analytiques avec KPIs et graphiques pour piloter la stratégie RH.

1.3 Étude de l'Existant

Avant de concevoir notre solution, nous avons analysé les outils existants sur le marché. Les principales conclusions sont les suivantes :

- Workday, SAP SuccessFactors : fonctionnels et complets, mais coûts de licence prohibitifs pour les PME (plusieurs milliers d'euros/an), et absence de personnalisation.
- BambooHR, Zoho Recruit : solutions intermédiaires accessibles, mais sans IA native pour l'analyse de CV et sans adaptation aux pratiques marocaines.

- OpenCATS, Orange HRM : open-source mais interface datée, peu de fonctionnalités IA, et maintenance communautaire incertaine.

✓ Positionnement de JobTech Solutions : solution sur mesure combinant les fonctionnalités essentielles d'un ATS moderne avec des capacités IA avancées, développée en Python/Django — accessible, personnalisable et adaptée au contexte EMSI.

2. Organisation du Projet

2.1 Équipe Projet

L'équipe est composée de trois étudiants ingénieurs de l'EMSI, chacun apportant une expertise complémentaire :

Membre	Rôle	Responsabilités	Compétences
Diouri Mehdi	Chef de projet / Backend	Architecture Django, modèles ORM, auth RBAC, Celery, déploiement	Python, Django, MySQL, Git
El Kharrazi Ibtihal	Frontend / UX	Interfaces utilisateur, templates Django, AJAX, tableaux de bord	HTML5, CSS3, JS, Bootstrap
Assaadi Mohamed Nadir	IA / Data Science	Pipeline NLP/ML, scoring CV, rapports PDF, graphiques	SpaCy, BERT, Scikit-learn, ReportLab

2.2 Planification par Sprint (Agile)

Le développement suit une méthodologie Agile avec des sprints de 1 à 2 semaines. Chaque sprint suit le cycle : Planification → Développement → Revue → Rétrospective. Les statuts sont mis à jour en temps réel (☑ Terminé, 🔄 En cours, ☐ Planifié) :

Sprint	Durée	Thème	Objectifs	Responsable	État
S1	2 sem.	Initialisation	Env. Django, BDD MySQL, Auth, RBAC, middleware audit	Diouri M.	☑ Terminé
S2	2 sem.	Gestion Offres	CRUD offres CBV, formulaires, filtres, tags compétences	El Kharrazi I.	☑ Terminé
S3	2 sem.	Candidatures	Upload CV, pipeline déclenchement, dashboard candidat	El Kharrazi I.	☑ Terminé
S4	2 sem.	NLP Parser	SpaCy/BERT, extraction entités, normalisation, insertion BDD	Assaadi M.N.	🔄 En cours
S5	2 sem.	ML Scorer + Entretiens	Scoring TF-IDF, jauge, calendrier, notifications Celery	Assaadi+Diouri	🔄 En cours
S6	2 sem.	Évaluations + Notes	Formulaires éval, AJAX temps réel, PDF ReportLab	Assaadi+El K.	☐ Planifié
S7	2 sem.	Dashboard RH	KPIs, graphiques Matplotlib, filtres analytiques	El Kharrazi I.	☐ Planifié
S8	1 sem.	Déploiement	Gunicorn/Nginx, HTTPS Let's Encrypt, sauvegardes cron	Diouri M.	☐ Planifié

S9-10	2 sem.	Tests & Docs	Tests unitaires, intégration, rapport final	Équipe	☐ Planifié
--------------	-----------	-------------------------	---	--------	-----------------------------------

2.3 Budget Global

Le budget total alloué au projet s'élève à 20 000 €, réparti comme suit. Le développement représente la part la plus importante (50%), reflétant la priorité accordée à la qualité du code et à l'implémentation des fonctionnalités IA :

Poste	Détail	Montant	Part
Développement	Salaires dev backend, frontend, IA (3 développeurs × durée projet)	10 000 €	50%
Matériel	Serveurs VPS, postes de travail, équipements réseau	5 000 €	25%
Maintenance	Corrections bugs, MAJ sécurité, support 1 an post-livraison	3 000 €	15%
Licences	Hébergement cloud, domaine, outils CI/CD, licences logicielles	2 000 €	10%
TOTAL		20 000 €	100%

✦ *Ce chapitre a posé les bases du projet : contexte, problématique, objectifs et organisation. La fragmentation des outils RH existants justifie le développement d'une solution centralisée, intelligente et sur mesure. Le chapitre suivant présente la conception technique détaillée.*

Chapitre 2 : Conception Technique

Introduction

Ce chapitre constitue le cœur technique du projet. Il présente l'architecture détaillée par couches, le modèle de classes UML complet avec méthodes et cardinalités, le schéma ERD de la base de données MySQL, et la répartition précise des tâches de développement. L'objectif est de fournir une base de conception suffisamment précise pour guider l'implémentation et garantir la cohérence technique entre les membres de l'équipe.

1. Architecture Technique Détaillée

1.1 Vue Globale des Couches

JobTech Solutions est structuré en cinq couches indépendantes et spécialisées, conformément au principe de séparation des préoccupations (Separation of Concerns). Cette architecture garantit la maintenabilité, la testabilité et l'évolutivité du système :

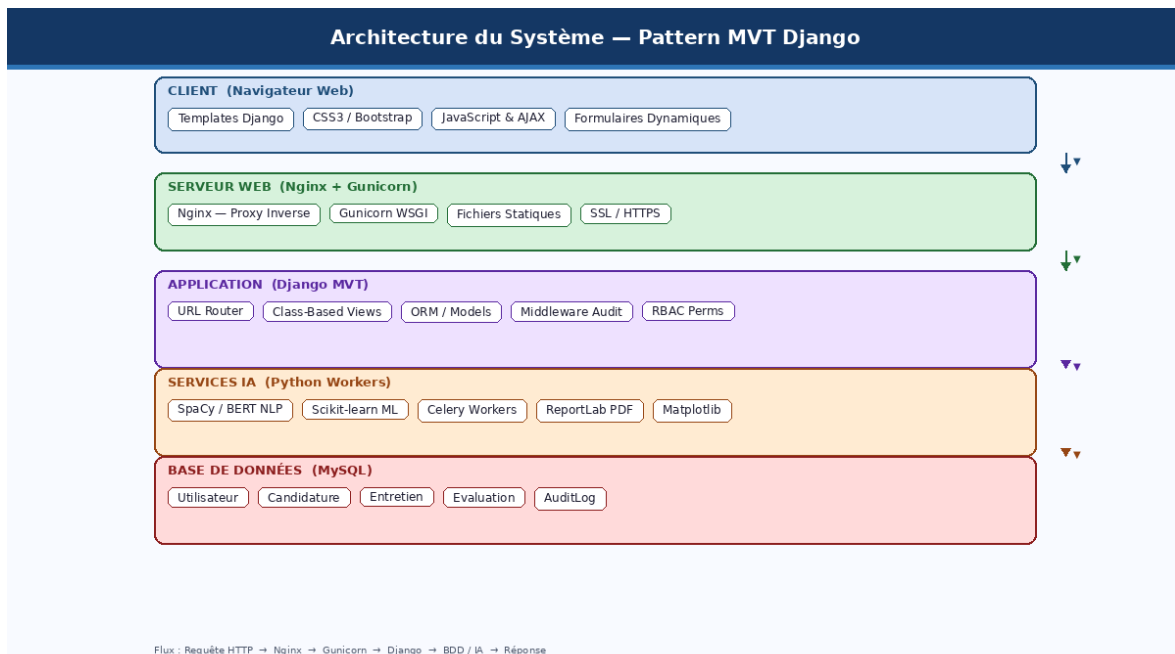


Figure 3 : Architecture du système — Pattern MVT Django, couches et flux

De haut en bas, les cinq couches sont :

- **Couche Présentation (Frontend)** : templates Django côté serveur, CSS3/Bootstrap pour le responsive design, JavaScript et AJAX pour les interactions asynchrones. Aucune logique métier dans cette couche.
- **Couche Serveur Web (Nginx + Gunicorn)** : Nginx reçoit toutes les requêtes HTTP/HTTPS, sert les fichiers statiques directement (CSS, JS, images) pour des raisons de performance, et transmet les requêtes dynamiques à Gunicorn. Gunicorn instancie plusieurs workers Python pour le traitement parallèle.
- **Couche Application (Django MVT)** : cœur de l'application. L'URL Router dispatche vers les vues CBV. Les vues appliquent les permissions RBAC, interagissent avec l'ORM pour accéder à MySQL, et préparent les données pour les templates. Le middleware d'audit intercepte toutes les opérations CRUD.

- Couche Services IA (Python Workers) : découplée du serveur via Celery. Les workers s'exécutent en processus indépendants pour les traitements lourds (NLP, ML, génération PDF) sans bloquer le serveur principal. Communication via broker Redis.
- Couche Données (MySQL) : stockage persistant via ORM Django. Accès exclusivement par la couche applicative. Index définis sur les colonnes de filtrage fréquent. Sauvegardes automatiques quotidiennes.

1.2 Composants Internes par Couche

La couche Application Django est la plus riche. On y distingue six composants principaux :

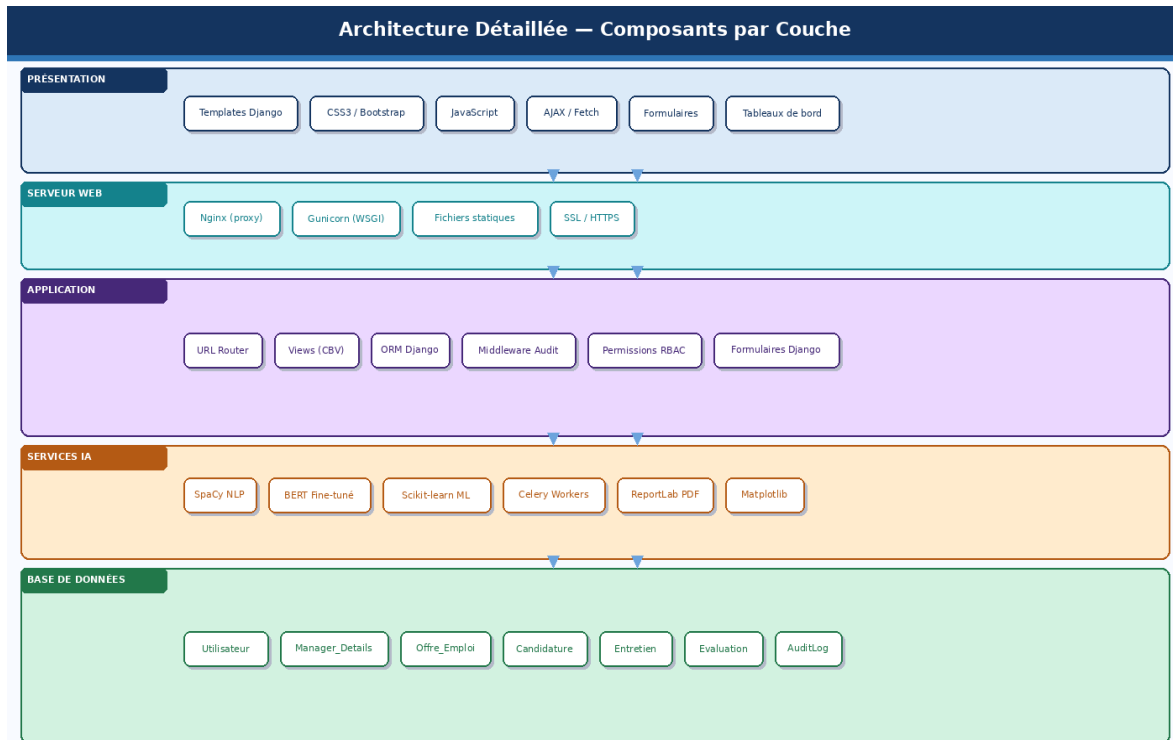


Figure 5 : Architecture technique détaillée — Tous les composants par couche

2. Modèle de Classes UML

2.1 Description des Classes et Méthodes

Le diagramme de classes représente la modélisation objet complète de JobTech Solutions. Chaque classe encapsule ses attributs (avec types SQL), sa visibilité (+ public, – privé) et ses méthodes métier. La classe Utilisateur, centrale, hérite d'AbstractUser Django pour bénéficier nativement de l'authentification et du hachage des mots de passe (PBKDF2) :

Diagramme de Classes — Modèle de Données MySQL

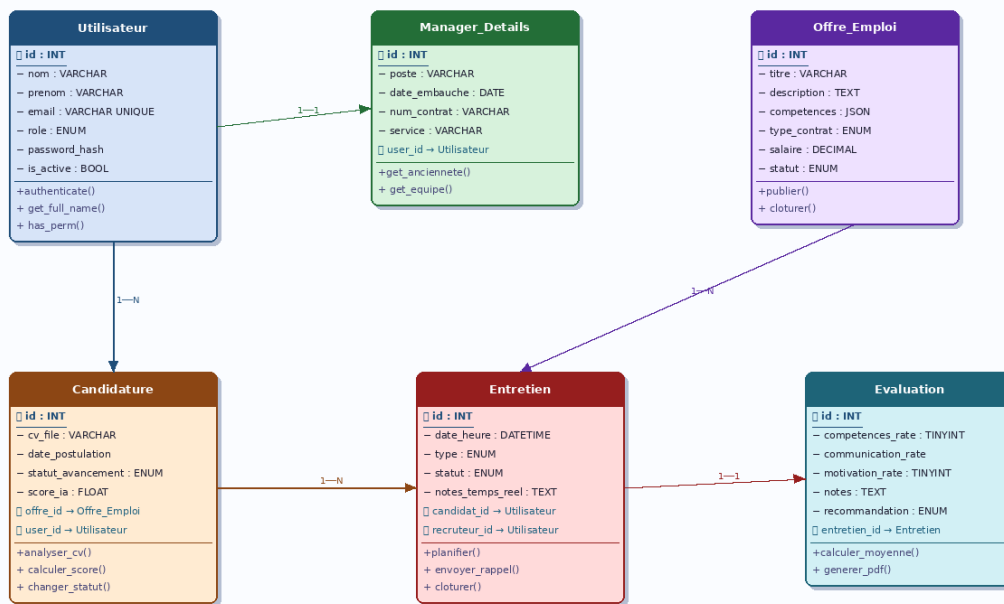


Figure 4 : Diagramme de classes — Modèle de données avec relations

Diagramme de Classes UML — JobTech Solutions

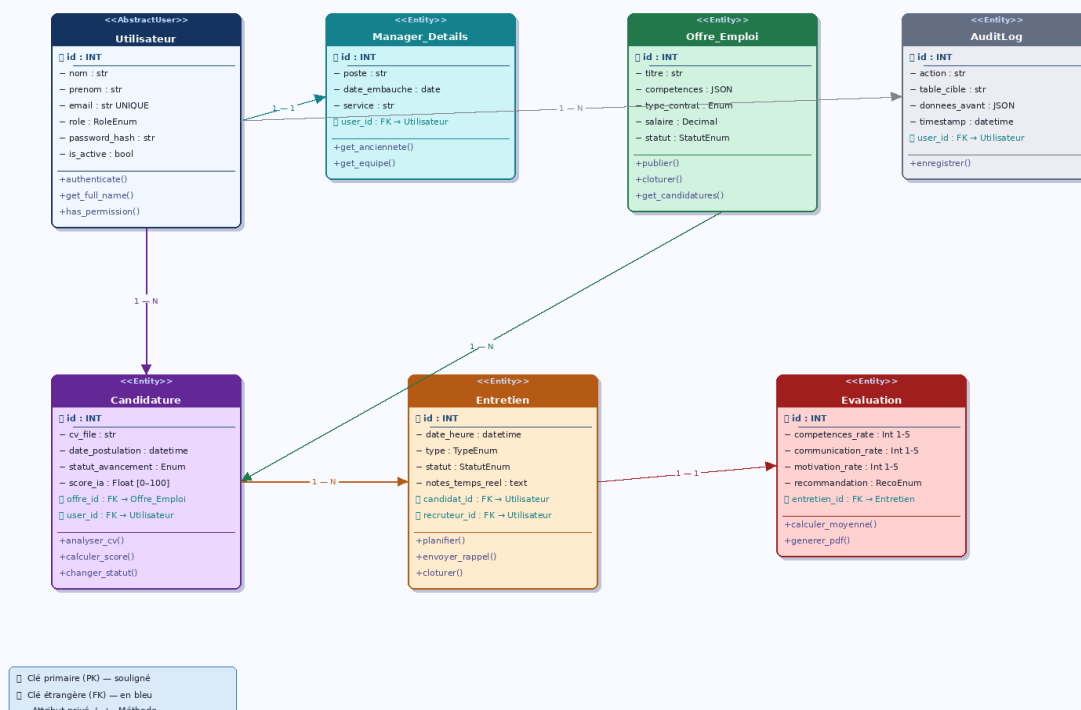


Figure 6 : Diagramme de classes UML complet avec méthodes et stéréotypes

2.2 Relations et Cardinalités

Les relations entre classes suivent des cardinalités strictes reflétant les règles métier :

- Utilisateur ↔ Manager_Details (1..1) : OneToOneField Django avec cascade. Chaque manager a exactement un profil détaillé.

- Utilisateur → Candidature (1..N) : un candidat peut soumettre plusieurs candidatures, une par offre.
- Offre_Emploi → Candidature (1..N) : une offre peut recevoir N candidatures, chacune ne cible qu'une offre.
- Candidature → Entretien (1..N) : une candidature peut donner lieu à plusieurs entretiens (RH, technique, final).
- Entretien → Evaluation (1..1) : obligation métier — tout entretien clôturé doit avoir une évaluation associée.
- Utilisateur → AuditLog (1..N) : toutes les actions sont loggées, même après suppression du compte.

3. Schéma de la Base de Données (ERD)

3.1 Modèle Entité-Relation

Le schéma ERD définit la structure physique de la base MySQL avec les types SQL exacts, les contraintes (NOT NULL, UNIQUE, DEFAULT) et les clés étrangères. L'encodage utf8mb4 supporte tous les caractères Unicode. Le moteur InnoDB garantit les transactions ACID et les contraintes FK :

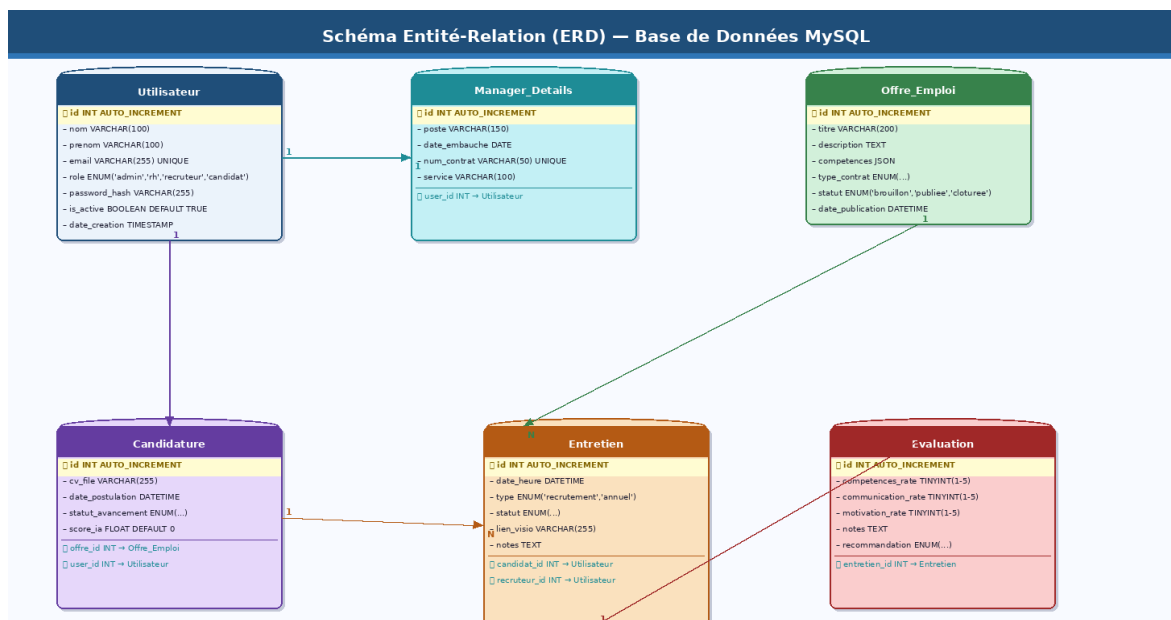


Figure 7 : Schéma Entité-Relation (ERD) — Base de données MySQL

3.2 Contraintes et Index

Les index sont définis sur les colonnes les plus sollicitées pour optimiser les performances :

- Utilisateur : UNIQUE sur email. Index composite (role, is_active) pour filtrer les utilisateurs actifs par rôle.
- Candidature : Index composite (offre_id, statut_avancement) pour les listes triées. Index sur score_ia pour le classement décroissant.
- Entretien : Index sur (date_heure, statut) pour les plannings. Index sur recruteur_id pour les agendas individuels.

- AuditLog : Index sur (user_id, timestamp) pour l'historique chronologique. CHECK CONSTRAINT sur les ratings ($1 \leq \text{valeur} \leq 5$).
- ON DELETE CASCADE sur Candidature.user_id et Entretien.candidat_id pour le nettoyage automatique des données orphelines.

4. Allocation des Tâches par Module

4.1 Répartition par Membre

Chaque membre est responsable d'un domaine d'expertise clair, avec des points de synchronisation hebdomadaires (stand-up 15 min) et des Pull Requests GitHub pour la revue de code. Toute modification d'architecture est validée collectivement avant merge sur la branche develop :

Membre	Rôle	Responsabilités	Compétences
Diouri Mehdi	Chef de projet / Backend	Architecture Django, modèles ORM, auth RBAC, Celery, déploiement	Python, Django, MySQL, Git
El Kharrazi Ibtihal	Frontend / UX	Interfaces utilisateur, templates Django, AJAX, tableaux de bord	HTML5, CSS3, JS, Bootstrap
Assaadi Mohamed Nadir	IA / Data Science	Pipeline NLP/ML, scoring CV, rapports PDF, graphiques	SpaCy, BERT, Scikit-learn, ReportLab

4.2 Tableau Détaillé par Module

Le tableau suivant détaille les 10 modules fonctionnels avec leurs tâches spécifiques, le responsable, la charge estimée, la priorité (● Haute / ● Moyenne / ● Basse) et le sprint cible. La charge totale estimée est de ~40 jours-homme répartis équitablement :

Module	Tâches	Responsable	Charge	Priorité	Sprint
Auth & Sécurité	- Modèle Utilisateur AbstractUser - RBAC 4 rôles - Middleware audit CRUD - Décorateurs @login_required	Diouri Mehdi	3j	● Haute	S1
Offres d'Emploi	- CRUD Offres (CBV Django) - Tags compétences (JSON) - Filtres & recherche - Gestion statuts	El Kharrazi I.	4j	● Haute	S2
Candidatures	- Upload CV (PDF/DOCX) - Stockage + chemin BDD - Suivi statut avancement - Dashboard candidat	El Kharrazi I.	4j	● Haute	S3
NLP Parser	- SpaCy/BERT extraction - Nom, compétences, exp. - Normalisation données - Insertion BDD	Assaadi M.N.	5j	● Haute	S4

ML Scorer	- Vectorisation TF-IDF - Similarité cosinus - Score 0–100% - Jauge colorée UI	Assaadi M.N.	4j	● Haute	S5
Entretiens	- CRUD planification - Calendrier FullCalendar - Notifs email Celery - Rappels J-1 et H-2	Diouri Mehdi	4j	● Moyenne	S5
Évaluations & PDF	- Formulaire structuré - Notation critères 1–5 - PDF ReportLab - Graphiques Matplotlib	Assaadi M.N.	4j	● Moyenne	S6
Notes Temps Réel	- Sauvegarde AJAX 30s - Interface prise de notes - Historique par entretien	El Kharrazi I.	2j	● Moyenne	S6
Dashboard RH	- KPIs recrutement - Graphiques dynamiques - Filtres analytiques - Export CSV	El Kharrazi I.	3j	● Basse	S7
Déploiement	- Gunicorn + Nginx - HTTPS Let's Encrypt - Sauvegardes MySQL - Requirements.txt	Diouri Mehdi	3j	● Basse	S8

4.3 Planification par Sprint

9 sprints couvrent l'intégralité du projet sur 18 semaines. Les sprints S1-S3 sont critiques : leur réussite conditionne tous les développements suivants. Les sprints S4-S5 (modules IA) sont les plus complexes techniquement :

Sprint	Durée	Thème	Objectifs	Responsable	État
S1	2 sem.	Initialisation	Env. Django, BDD MySQL, Auth, RBAC, middleware audit	Diouri M.	☑ Terminé
S2	2 sem.	Gestion Offres	CRUD offres CBV, formulaires, filtres, tags compétences	El Kharrazi I.	☑ Terminé
S3	2 sem.	Candidatures	Upload CV, pipeline déclenchement, dashboard candidat	El Kharrazi I.	☑ Terminé
S4	2 sem.	NLP Parser	SpaCy/BERT, extraction entités, normalisation, insertion BDD	Assaadi M.N.	🔄 En cours
S5	2 sem.	ML Scorer + Entretiens	Scoring TF-IDF, jauge, calendrier, notifications Celery	Assaadi+Diouri	🔄 En cours
S6	2 sem.	Évaluations + Notes	Formulaires éval, AJAX temps réel, PDF ReportLab	Assaadi+El K.	☐ Planifié
S7	2 sem.	Dashboard RH	KPIs, graphiques Matplotlib, filtres analytiques	El Kharrazi I.	☐ Planifié
S8	1 sem.	Déploiement	Gunicorn/Nginx, HTTPS Let's Encrypt, sauvegardes cron	Diouri M.	☐ Planifié

S9-10	2 sem.	Tests & Docs	Tests unitaires, intégration, rapport final	Équipe	☐ Planifié
-------	-----------	--------------	---	--------	--------------------------------------

✦ *Ce chapitre a fourni la conception technique complète : architecture 5 couches, classes UML détaillées, ERD MySQL avec contraintes, et allocation précise des tâches. Ces spécifications servent de référence directe pour l'implémentation dans les sprints suivants.*

Chapitre 3 : Analyse et Conception

Introduction

Ce chapitre présente l'analyse des besoins fonctionnels et non fonctionnels, les choix technologiques justifiés, et la modélisation UML comportementale de la solution. Les diagrammes des cas d'utilisation et de séquence illustrent les interactions entre acteurs et composants.

1. Description des Besoins Fonctionnels

1.1 Périmètre et Acteurs

Le système interagit avec quatre acteurs distincts, chacun disposant d'un espace fonctionnel dédié adapté à ses responsabilités et à son niveau d'accès RBAC :

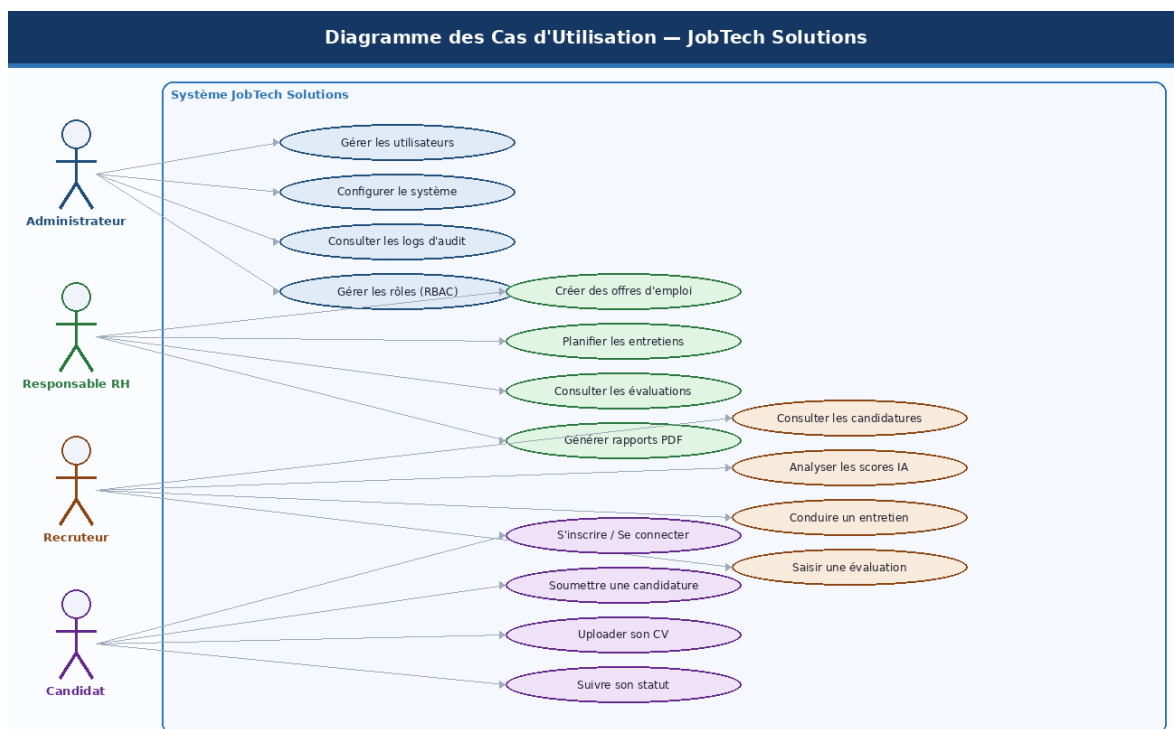


Figure 1 : Diagramme des cas d'utilisation — Les 4 acteurs et leurs fonctionnalités

1.2 Description des Modules Fonctionnels

► Module 1 — Authentification et Sécurité

Socle sécuritaire de l'application. Gestion du cycle de vie des comptes utilisateurs avec RBAC à 4 niveaux. L'authentification repose sur les sessions Django (PBKDF2). Un middleware personnalisé enregistre toutes les opérations CRUD dans une table AuditLog (user_id, action, table, données avant/après, timestamp, IP).



Rôles définis : Administrateur (accès total), Responsable RH (offres + évaluations), Recruteur (candidatures + entretiens), Candidat (profil + candidatures uniquement).

► Module 2 — Gestion des Offres d'Emploi

Les Responsables RH créent et publient des offres avec titre, description, compétences requises (format JSON/tags), type de contrat, salaire et statut (brouillon/publiée/clôturée). Les compétences définies ici alimentent directement l'algorithme de scoring IA du module ML.

► Module 3 — Candidatures et Pipeline IA

Les candidats soumettent leur CV (PDF ou DOCX). Dès l'upload, un worker Celery déclenche automatiquement le pipeline IA : (1) extraction du texte brut, (2) parsing NLP SpaCy/BERT pour extraire les entités, (3) vectorisation et calcul du score de compatibilité, (4) mise à jour du score_ia en base. Le recruteur voit alors le score 0–100% avec une jauge colorée.

► Module 4 — Planification des Entretiens

CRUD complet avec calendrier interactif (FullCalendar.js). Les notifications email sont envoyées automatiquement via Celery à la création, puis des rappels à J-1 et H-2 avant l'entretien. L'interface de conduite inclut une prise de notes sauvegardée toutes les 30 secondes par AJAX.

► Module 5 — Évaluations et Rapports PDF

Fiche d'évaluation standardisée avec notation 1–5 par critère (compétences techniques, communication, motivation), champs de commentaires libres, et recommandation finale (Retenu / À reconsidérer / Non retenu). Génération automatique d'un rapport PDF formaté (ReportLab) avec graphiques de performance (Matplotlib).

► Module 6 — Tableaux de Bord RH

KPIs calculés dynamiquement : taux de conversion par étape du funnel, délai moyen de recrutement, distribution des scores IA, évolution des performances annuelles N vs N-1. Graphiques générés côté serveur (Matplotlib) et transmis en base64. Filtres par période, département, recruteur.

2. Spécification Détaillée des Cas d'Utilisation

Les tableaux suivants décrivent en détail chacun des cas d'utilisation identifiés dans le diagramme, pour les quatre acteurs du système. Chaque fiche précise l'acteur concerné, les préconditions nécessaires, le scénario nominal pas à pas, les scénarios alternatifs, et la postcondition attendue.

Acteur 1 — Administrateur

UC-01 — Gérer les utilisateurs (Administrateur)

Acteur	Administrateur
Préconditions	L'administrateur est connecté avec le rôle Admin.
Scénario nominal	1. L'administrateur accède au menu « Gestion des utilisateurs ». 2. Il consulte la liste paginée de tous les comptes (filtrée par rôle, statut). 3. Il clique sur « Créer un compte » et remplit le formulaire (nom, email, rôle). 4. Le système valide les données, hache le mot de passe (PBKDF2) et enregistre en BDD. 5. Un email de confirmation avec lien d'activation est envoyé via Celery. 6. Le middleware d'audit enregistre l'action dans AuditLog.
Scénarios alternatifs	• Si l'email existe déjà : message d'erreur, formulaire non soumis. • Si l'email d'activation expire (48h) : l'admin peut renvoyer manuellement.
Postcondition	Le compte est créé, activé et apparaît dans la liste. L'utilisateur peut se connecter.

UC-02 — Gérer les rôles et permissions (Administrateur)

Acteur	Administrateur
---------------	----------------

Préconditions	L'administrateur est connecté. Le compte cible existe et est actif.
Scénario nominal	1. L'administrateur sélectionne un utilisateur dans la liste. 2. Il clique sur « Modifier le rôle ». 3. Il choisit parmi : Administrateur, Responsable RH, Recruteur, Candidat. 4. Il confirme la modification. 5. Le système met à jour le rôle en BDD et invalide la session courante de l'utilisateur. 6. L'action est enregistrée dans AuditLog avec l'ancien et le nouveau rôle.
Scénarios alternatifs	• Si l'admin tente de rétrograder le dernier compte Administrateur : refus avec message d'alerte. • Possibilité de désactiver temporairement un compte sans le supprimer.
Postcondition	Le rôle est mis à jour. L'utilisateur verra ses permissions modifiées à sa prochaine connexion.

UC-03 — Consulter les logs d'audit (Administrateur)

Acteur	Administrateur
Préconditions	L'administrateur est connecté.
Scénario nominal	1. L'administrateur accède au menu « Journal d'audit ». 2. Il applique des filtres optionnels : utilisateur, type d'action, table cible, période. 3. Le système affiche la liste paginée des entrées AuditLog correspondantes. 4. Chaque entrée affiche : date/heure, utilisateur, action, table, données avant/après, IP. 5. L'admin peut exporter les logs filtrés au format CSV pour audit externe.
Scénarios alternatifs	• Si aucun log ne correspond aux filtres : affichage d'un message « Aucun résultat ».
Postcondition	Les logs sont affichés. L'export CSV est généré et téléchargé.

UC-04 — Configurer le système (Administrateur)

Acteur	Administrateur
Préconditions	L'administrateur est connecté.
Scénario nominal	1. L'administrateur accède au menu « Configuration ». 2. Il peut modifier : critères d'évaluation par département, seuils d'alerte scoring IA (% , couleurs), templates d'emails (HTML), paramètres de notifications Celery. 3. Il sauvegarde les modifications. 4. Le système valide et applique la configuration immédiatement (pas besoin de redémarrage).
Scénarios alternatifs	• Si un template HTML est malformé : erreur de validation avec indication de la ligne incorrecte.
Postcondition	La configuration est sauvegardée et active immédiatement pour tous les utilisateurs.

Acteur 2 — Responsable RH

UC-05 — Créer une offre d'emploi *(Responsable RH)*

Acteur	Responsable RH
Préconditions	Le Responsable RH est connecté avec le rôle RH.
Scénario nominal	1. Le RH accède au menu « Offres d'emploi » → « Nouvelle offre ». 2. Il remplit le formulaire : titre du poste, description (éditeur rich text), compétences requises (tags JSON), type de contrat (CDI/CDD/Stage), fourchette salariale, date de clôture. 3. Il choisit le statut : Brouillon (non visible) ou Publiée (visible aux candidats). 4. Il clique sur « Enregistrer ». 5. Le système valide, enregistre en BDD et, si Publiée, rend l'offre accessible sur l'espace candidat. 6. Les tags de compétences sont indexés pour être utilisés par le module de scoring IA.
Scénarios alternatifs	• Si des champs obligatoires sont manquants : validation côté client et côté serveur avec messages d'erreur précis. • Le RH peut sauvegarder en Brouillon et reprendre plus tard.
Postcondition	L'offre est enregistrée. Si publiée, elle apparaît dans l'espace candidat.

UC-06 — Planifier les entretiens annuels *(Responsable RH)*

Acteur	Responsable RH
Préconditions	Le RH est connecté. Des comptes Manager_Details existent pour les collaborateurs concernés.
Scénario nominal	1. Le RH accède au menu « Entretiens annuels » → « Planifier campagne ». 2. Il sélectionne la période de la campagne (ex. : T4 2025) et les départements concernés. 3. Le système affiche la liste des collaborateurs sans entretien planifié sur la période. 4. Le RH assigne chaque entretien à un recruteur/manager et choisit une date. 5. Il confirme la planification en masse ou entretien par entretien. 6. Celery envoie automatiquement des emails de convocation à chaque collaborateur et recruteur. 7. Des rappels automatiques sont programmés à J-7 et J-1.
Scénarios alternatifs	• Si un collaborateur est absent (congé, maladie) : possibilité de reprogrammer directement depuis le calendrier. • Si un recruteur est surchargé : le système affiche une alerte de charge.
Postcondition	Les entretiens sont planifiés. Les emails de convocation sont envoyés. Les rappels sont programmés.

UC-07 — Consulter et valider les évaluations *(Responsable RH)*

Acteur	Responsable RH
Préconditions	Le RH est connecté. Des évaluations ont été saisies par les recruteurs.
Scénario nominal	1. Le RH accède au menu « Évaluations » et filtre par période, département ou recruteur. 2. Il consulte les fiches d'évaluation soumises.

Scénarios alternatifs	3. Pour chaque évaluation, il peut : valider (statut → Validée), demander une révision (statut → À réviser) avec commentaire, ou archiver. 4. Il peut générer un rapport PDF individuel ou un rapport agrégé par département. 5. Le rapport PDF est généré par ReportLab avec les graphiques Matplotlib inclus. • Si le recruteur n'a pas encore complété l'évaluation : statut « En attente » avec alerte.
Postcondition	Les évaluations sont validées. Les rapports PDF sont générés et téléchargeables.

UC-08 — Générer des rapports et statistiques (Responsable RH)

Acteur	Responsable RH
Préconditions	Le RH est connecté. Des données suffisantes existent (candidatures, entretiens, évaluations).
Scénario nominal	1. Le RH accède au menu « Tableau de bord ». 2. Il sélectionne les filtres : période, département, type d'entretien, recruteur. 3. Le système calcule les KPIs : taux de conversion par étape, délai moyen de recrutement, distribution des scores IA, top compétences recherchées. 4. Les graphiques sont générés dynamiquement par Matplotlib côté serveur. 5. Le RH peut exporter le tableau de bord complet en PDF ou les données brutes en CSV.
Scénarios alternatifs	• Si la période sélectionnée contient moins de 5 candidatures : avertissement sur la fiabilité statistique.
Postcondition	Le tableau de bord est affiché avec les KPIs. Les exports sont disponibles au téléchargement.

Acteur 3 — Recruteur

UC-09 — Consulter et trier les candidatures (Recruteur)

Acteur	Recruteur
Préconditions	Le recruteur est connecté avec le rôle Recruteur. Des candidatures existent pour ses offres.
Scénario nominal	1. Le recruteur accède au menu « Candidatures ». 2. Il filtre par offre, statut d'avancement, ou plage de score IA. 3. La liste affiche pour chaque candidature : nom du candidat, date de dépôt, score IA (jauge colorée), statut. 4. Il peut trier par score IA décroissant pour identifier les meilleurs profils en premier. 5. En cliquant sur un candidat, il accède à la fiche détaillée avec compétences extraites par NLP. 6. Il change le statut de la candidature (En attente → Présélectionné → Entretien planifié → Refusé).
Scénarios alternatifs	• Si le pipeline IA est encore en cours : affichage d'un loader avec message « Analyse en cours ». • Mode comparaison : sélection de 2 à 3 candidats côte à côte pour comparer les scores et compétences.

Postcondition	Les candidatures sont consultées et triées. Les statuts sont mis à jour en BDD.
----------------------	---

UC-10 — Analyser le score IA d'un candidat (Recruteur)

Acteur	Recruteur
Préconditions	Le recruteur est connecté. Le pipeline IA a été exécuté sur la candidature.
Scénario nominal	1. Le recruteur ouvre la fiche d'un candidat. 2. Il consulte le score de compatibilité global (0–100%) affiché sous forme de jauge colorée. 3. Il accède au détail du score par catégorie : Compétences techniques (%), Expérience (%), Formation (%), Langues (%). 4. Il consulte les compétences extraites automatiquement du CV par SpaCy/BERT. 5. Il peut comparer visuellement les compétences du candidat avec les exigences de l'offre via un radar chart. 6. Il prend sa décision de présélection basée sur ces données objectives.
Scénarios alternatifs	• Si le score IA est < 40% : alerte automatique suggérant de reconsidérer la candidature. • Le recruteur peut ignorer la suggestion IA et présélectionner manuellement le candidat.
Postcondition	Le recruteur a une vision complète et objective du profil candidat pour prendre sa décision.

UC-11 — Conduire un entretien (Recruteur)

Acteur	Recruteur
Préconditions	Le recruteur est connecté. Un entretien est planifié et son statut est « Planifié ».
Scénario nominal	1. Le recruteur accède au calendrier et clique sur l'entretien du jour. 2. Il passe le statut à « En cours ». 3. L'interface d'entretien s'ouvre avec : fiche du candidat, compétences IA, questions suggérées. 4. Il saisit ses notes en temps réel dans l'éditeur de notes. 5. Le système sauvegarde automatiquement les notes toutes les 30 secondes via AJAX (aucune perte de données possible). 6. À la fin, il clique sur « Clôturer l'entretien ». 7. Le statut passe à « Terminé » et le formulaire d'évaluation s'ouvre automatiquement.
Scénarios alternatifs	• Si le candidat est absent : le recruteur peut marquer l'entretien « Annulé » et reprogrammer. • Si la connexion internet est perdue : les notes sont sauvegardées localement (localStorage) et synchronisées à la reconnexion.
Postcondition	L'entretien est clôturé avec statut « Terminé ». Les notes sont sauvegardées. La fiche d'évaluation est ouverte.

UC-12 — Saisir une évaluation post-entretien (Recruteur)

Acteur	Recruteur
---------------	-----------

Préconditions	Le recruteur est connecté. L'entretien est terminé. La fiche d'évaluation est ouverte.
Scénario nominal	1. Le recruteur remplit la fiche d'évaluation structurée. 2. Il note chaque critère sur 5 : Compétences techniques, Communication, Motivation, Adaptabilité, Culture fit. 3. Il rédige un commentaire libre détaillé. 4. Il choisit sa recommandation finale : Retenu / À reconsidérer / Non retenu. 5. Il soumet la fiche. 6. Le système calcule la moyenne pondérée des critères. 7. ReportLab génère automatiquement le rapport PDF d'évaluation avec les graphiques Matplotlib. 8. Le rapport est envoyé par email au Responsable RH via Celery.
Scénarios alternatifs	• Si des critères obligatoires ne sont pas remplis : validation bloquante avec mise en évidence des champs manquants. • Le recruteur peut sauvegarder en brouillon et finaliser plus tard (délai max 48h).
Postcondition	L'évaluation est enregistrée. Le rapport PDF est généré et envoyé au RH. Le statut de la candidature est mis à jour.

Acteur 4 — Candidat

UC-13 — S'inscrire et se connecter (Candidat)

Acteur	Candidat
Préconditions	L'utilisateur n'a pas encore de compte (inscription) ou possède un compte actif (connexion).
Scénario nominal	1. Le candidat accède à la page d'accueil de JobTech Solutions. 2. Il clique sur « S'inscrire » et remplit le formulaire (nom, prénom, email, mot de passe). 3. Le système valide le format email (unicité), la force du mot de passe (min. 8 chars, 1 chiffre, 1 majuscule). 4. Un email de confirmation avec lien d'activation (valable 48h) est envoyé via Celery. 5. Le candidat clique sur le lien → compte activé → redirection vers la page de connexion. 6. Lors de la connexion : soumission email/mot de passe, vérification PBKDF2, création de session Django sécurisée.
Scénarios alternatifs	• Si le mot de passe est oublié : workflow de réinitialisation par email avec token CSRF unique (valable 1h). • Si 5 tentatives de connexion échouent : blocage temporaire du compte (15 min) pour éviter le brute-force.
Postcondition	Le compte candidat est créé et actif. La session Django est ouverte. Le candidat accède à son dashboard.

UC-14 — Soumettre une candidature (Candidat)

Acteur	Candidat
Préconditions	Le candidat est connecté. Au moins une offre d'emploi est publiée.

Scénario nominal	1. Le candidat accède à « Offres disponibles » et consulte la liste des offres publiées. 2. Il filtre par type de contrat, compétences requises, ou mots-clés. 3. Il ouvre la fiche d'une offre et clique sur « Postuler ». 4. Le formulaire de candidature s'ouvre en 3 étapes : (1) Vérification des informations personnelles, (2) Lettre de motivation optionnelle, (3) Upload du CV. 5. Le CV est uploadé (PDF ou DOCX, max 5 Mo). 6. Le système enregistre la candidature avec statut « Reçue » et déclenche automatiquement le worker Celery pour le pipeline IA. 7. Le candidat voit une barre de progression indiquant l'analyse IA en cours. 8. Une fois l'analyse terminée, son score de compatibilité s'affiche sur son dashboard.
Scénarios alternatifs	• Si le candidat a déjà postulé à cette offre : message d'erreur bloquant (une seule candidature par offre). • Si le fichier CV dépasse 5 Mo ou n'est pas PDF/DOCX : message d'erreur avec indication du format accepté. • Si le pipeline IA échoue (timeout) : notification email au candidat, possibilité de re-soumettre.
Postcondition	La candidature est enregistrée. Le pipeline IA est déclenché. Le candidat reçoit une confirmation par email.

UC-15 — Uploader et gérer son CV (Candidat)

Acteur	Candidat
Préconditions	Le candidat est connecté et possède un compte actif.
Scénario nominal	1. Le candidat accède à « Mon profil » → « Mon CV ». 2. Il uploadé un nouveau CV (PDF ou DOCX). 3. Le système remplace l'ancien CV, met à jour le chemin en BDD. 4. Le pipeline NLP est automatiquement re-exécuté sur le nouveau CV. 5. Les données extraites (compétences, expériences, formations) sont mises à jour en BDD. 6. Le candidat peut consulter un résumé des informations extraites automatiquement de son CV. 7. Si des erreurs d'extraction sont détectées, le candidat peut les corriger manuellement.
Scénarios alternatifs	• Si le format de CV est non standard (scan mal reconnu) : le système indique les champs non extraits. • Le candidat peut garder plusieurs versions de CV et choisir lequel associer à chaque candidature.
Postcondition	Le nouveau CV est enregistré. Les données du profil sont mises à jour via le pipeline NLP.

UC-16 — Suivre le statut de ses candidatures (Candidat)

Acteur	Candidat
Préconditions	Le candidat est connecté. Il a soumis au moins une candidature.
Scénario nominal	1. Le candidat accède à son dashboard « Mes candidatures ». 2. Il voit la liste de toutes ses candidatures avec pour chacune : titre de l'offre, date de dépôt, score IA (jauge colorée), statut d'avancement sur un stepper visuel.

Scénarios alternatifs	3. Le stepper affiche les étapes : ① Reçue → ② Analyse IA → ③ Examen RH → ④ Entretien planifié → ⑤ Décision finale. 4. Le candidat clique sur une candidature pour voir le détail : notes du recruteur (si partagées), date d'entretien planifiée, feedback éventuel. 5. Il reçoit automatiquement des emails de notification à chaque changement de statut. • Si son entretien est planifié : notification email avec date, heure, lieu et lien visioconférence. • Si sa candidature est refusée : notification email avec feedback générique (le détail de l'évaluation reste confidentiel).
Postcondition	Le candidat a une visibilité complète et en temps réel sur l'avancement de toutes ses candidatures.

ID	Cas d'utilisation	Acteur	Priorité	Sprint
UC-01	Gérer les utilisateurs	Administrateur	● Haute	S1
UC-02	Gérer les rôles	Administrateur	● Haute	S1
UC-03	Consulter les logs	Administrateur	● Moyenne	S7
UC-04	Configurer le système	Administrateur	● Basse	S8
UC-05	Créer une offre	Responsable RH	● Haute	S2
UC-06	Planifier entretiens annuels	Responsable RH	● Moyenne	S5
UC-07	Valider les évaluations	Responsable RH	● Moyenne	S6
UC-08	Générer rapports/stats	Responsable RH	● Basse	S7
UC-09	Consulter les candidatures	Recruteur	● Haute	S3
UC-10	Analyser le score IA	Recruteur	● Haute	S4-S5
UC-11	Conduire un entretien	Recruteur	● Haute	S5
UC-12	Saisir une évaluation	Recruteur	● Haute	S6
UC-13	S'inscrire / Se connecter	Candidat	● Haute	S1
UC-14	Soumettre une candidature	Candidat	● Haute	S3
UC-15	Uploader son CV	Candidat	● Haute	S3-S4

UC-16	Suivre ses candidatures	Candidat	 Moyenne	S3
-------	-------------------------	----------	---	----

3. Stack Technologique

Le choix des technologies a été guidé par la maîtrise de l'équipe, la maturité des bibliothèques, les performances et la cohérence de l'écosystème Python :

Technologie	Catégorie	Rôle et justification	Usage
Python 3.x	Langage	Backend + scripts IA. Typage dynamique, riche en bibliothèques scientifiques.	Tout le projet
Django 4.x	Framework	Pattern MVT, ORM intégré, sécurité native, admin auto-généré.	Backend web
MySQL 8.x	SGBD	Base relationnelle, transactions ACID, indexation performante.	Données
SpaCy	NLP	Extraction d'entités nommées (NER) depuis les CV. Modèle fr_core_news.	Module IA
BERT (HuggingFace)	NLP avancé	Embeddings contextuels pour extraction compétences et matching sémantique.	Module IA
Scikit-learn	ML	TF-IDF, similarité cosinus, scoring de compatibilité candidat/offre.	Module IA
Celery + Redis	Async	Traitement IA en arrière-plan. Notifications email planifiées.	Tâches async
ReportLab	PDF	Génération rapports évaluations personnalisés, export professionnel.	Rapports
Matplotlib	Graphiques	Visualisation KPIs RH, graphiques de performance côté serveur.	Dashboard
Nginx + Unicorn	Serveur	Proxy inverse + WSGI. Architecture production stable et performante.	Déploiement

4. Modélisation UML Comportementale

4.1 Diagramme de Séquence — Pipeline IA

Le scénario le plus complexe du système est l'upload d'un CV et l'exécution du pipeline IA complet. Ce diagramme illustre les 11 interactions entre les 6 composants du système, en mettant en évidence le traitement asynchrone via Celery (étapes 5 à 9) :

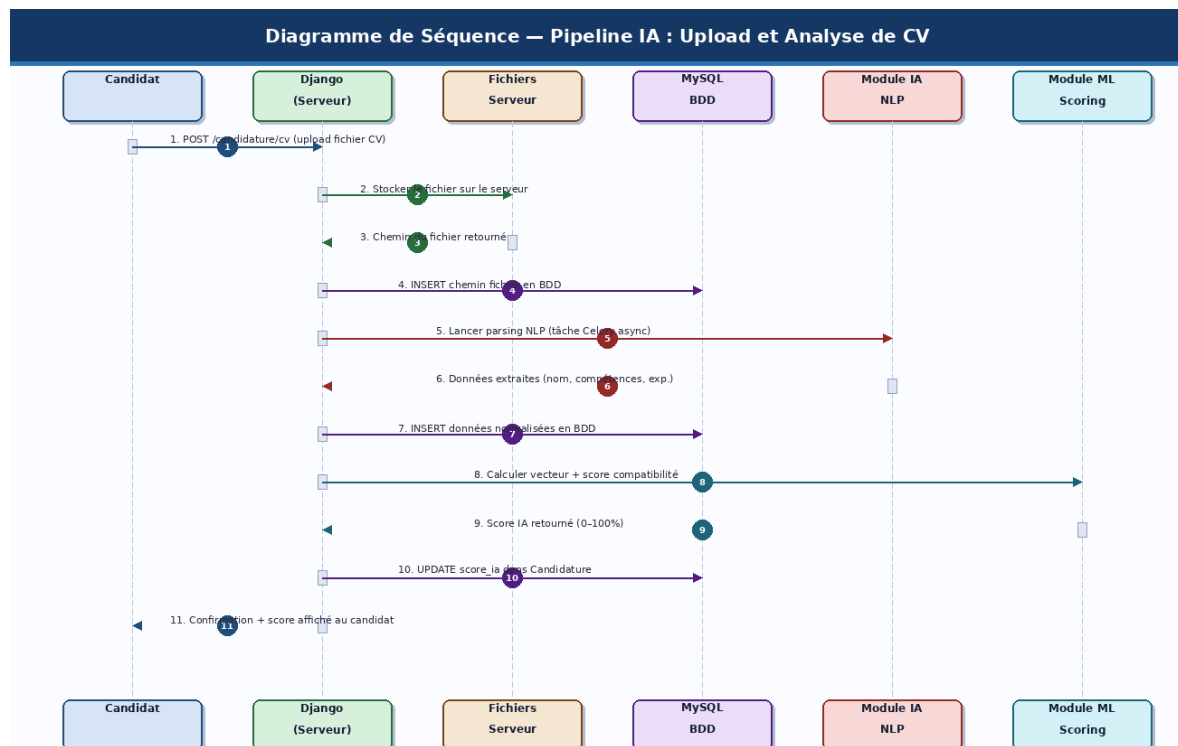


Figure 2 : Diagramme de séquence — Pipeline IA : Upload CV, NLP, ML, Score

5. Exigences Non Fonctionnelles

Au-delà des fonctionnalités, le système doit respecter des exigences de qualité strictes pour garantir son utilisation en production :

Exigence	Description
Performance	< 2s pour les requêtes classiques. Les traitements IA (parsing NLP, scoring ML) s'exécutent via des workers Celery en arrière-plan pour ne pas bloquer l'interface. Un loader visuel informe l'utilisateur pendant l'analyse.
Sécurité	Toutes les routes sont protégées par <code>@login_required</code> et <code>@permission_required</code> . Protection CSRF et XSS native Django. Mots de passe hachés PBKDF2. Middleware d'audit traçant chaque opération CRUD avec user, timestamp, données avant/après.
Compatibilité	Interface responsive compatible Chrome, Firefox, Edge, Safari. Design Bootstrap adapté desktop et tablette. Dégradation gracieuse sur navigateurs moins récents.
Disponibilité	Cible 99% en production via Unicorn (workers multiples) + Nginx. Sauvegardes MySQL automatiques quotidiennes via cron. Monitoring via Supervisor pour les workers Celery.
Maintenabilité	Architecture MVT stricte, code documenté (docstrings), séparation modules (web/IA/rapports). Versioning Git Flow : main/develop/feature/*. Migrations Django versionnées pour l'évolution du schéma.

6. Diagramme d'État — Cycle de Vie d'une Candidature

Le diagramme d'état modélise les différents états possibles d'une candidature tout au long de son cycle de vie, ainsi que les transitions déclenchées par les actions des acteurs ou du système :

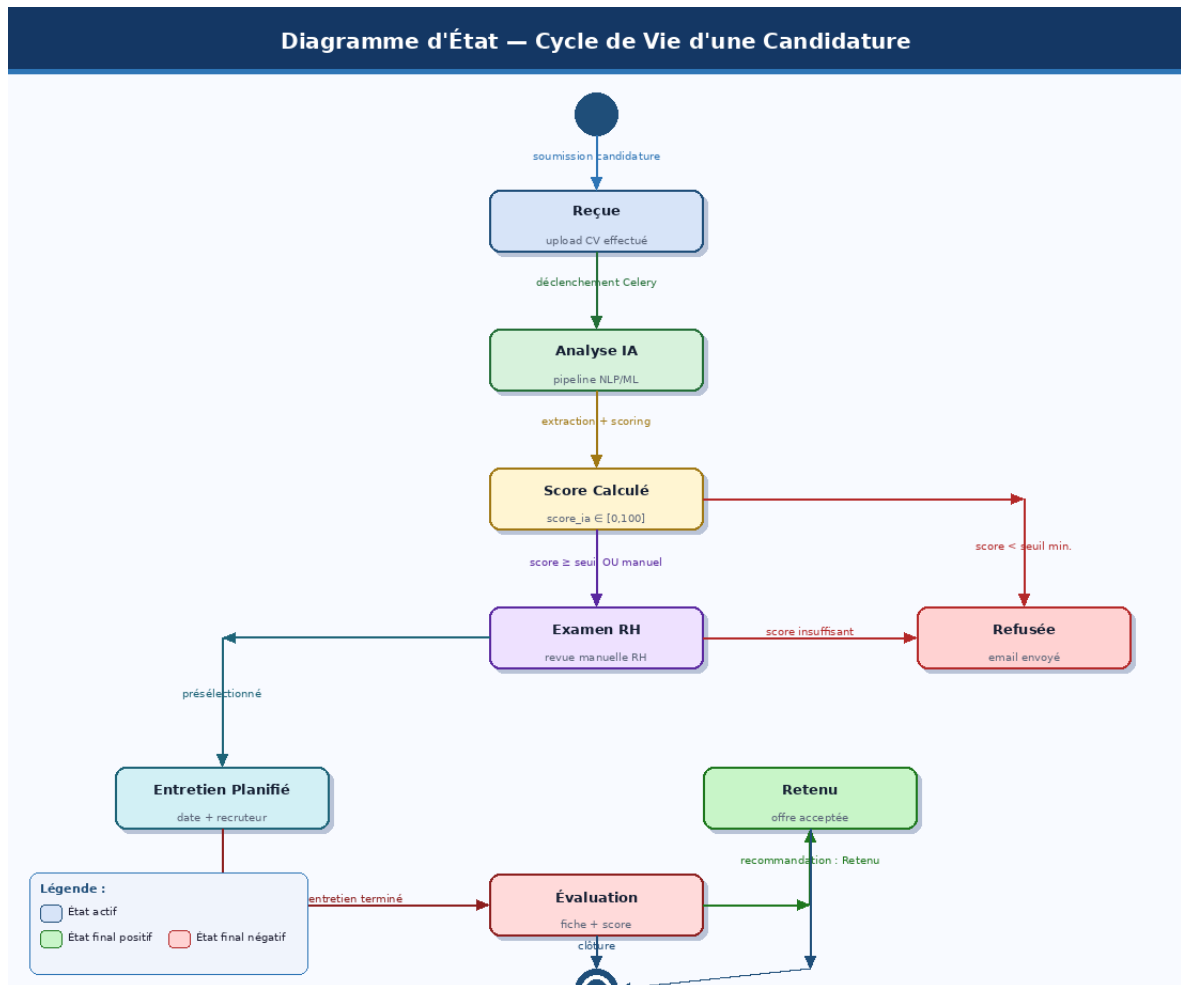


Figure 8 : Diagramme d'état — Cycle de vie d'une candidature

Six états principaux sont définis : Reçue (après upload CV), Analyse IA (pipeline NLP/ML en cours), Score Calculé (score_ia mis à jour), Examen RH (revue manuelle), Entretien Planifié, Évaluation. Deux états finaux : Retenu (recommandation positive) et Refusée (score insuffisant ou décision négative).

7. Diagramme d'Activité — Processus de Recrutement

Le diagramme d'activité représente le flux complet du processus de recrutement avec ses swim lanes pour chaque acteur. Il montre la collaboration et les transitions de responsabilité :

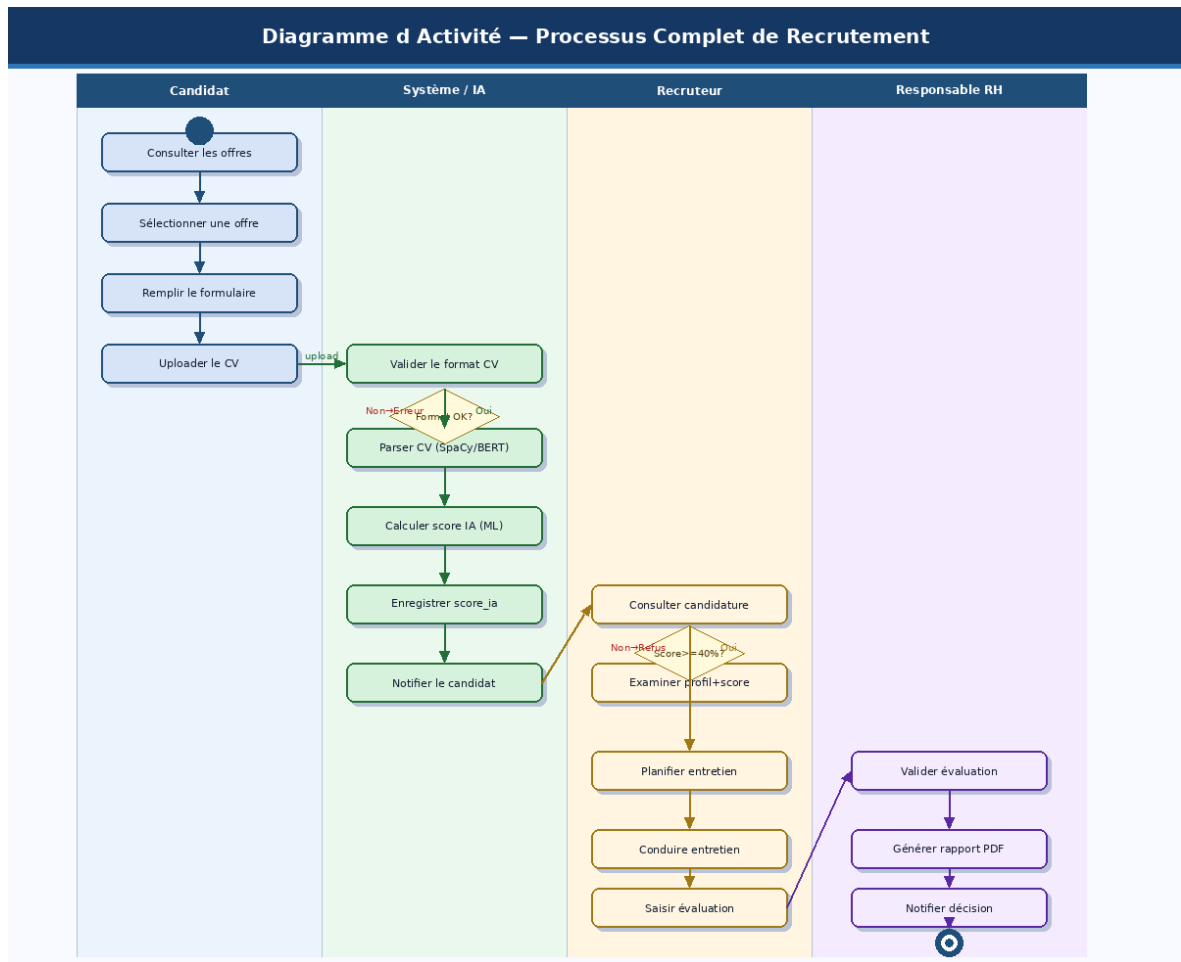


Figure 9 : Diagramme d'activité — Processus complet de recrutement

✦ Ce chapitre a couvert l'analyse fonctionnelle complète (6 modules + 16 cas d'utilisation), le diagramme d'état du cycle de vie d'une candidature, le diagramme d'activité du processus de recrutement, les choix technologiques justifiés et les exigences non fonctionnelles.

Chapitre 4 : Réalisation et Interfaces

Introduction

Ce chapitre présente les résultats concrets du développement : infrastructure de déploiement, maquettes des interfaces principales, et détail de l'implémentation des fonctionnalités clés (pipeline IA NLP/ML, génération de rapports, sauvegarde temps réel).

1. Infrastructure et Déploiement

1.1 Environnements



Développement (local) : instance Django locale avec base MySQL dédiée. Configuration via `.env` (`DEBUG=True`). Variables d'environnement pour éviter les secrets dans le code versionné.



Production (VPS Linux) : Gunicorn (4 workers) derrière Nginx. HTTPS via Let's Encrypt. MySQL avec pool de connexions. Supervisor pour les workers Celery.

1.2 Pipeline de Déploiement Git Flow

Les fonctionnalités sont développées sur des branches `feature/*`, mergées via Pull Request vers `develop` après revue de code, puis déployées sur `main`. Les migrations Django s'appliquent automatiquement à chaque déploiement.

2. Maquettes des Interfaces (Wireframes)

Les maquettes suivantes représentent le design fonctionnel des interfaces principales. Ces wireframes haute fidélité illustrent la disposition des éléments, la navigation, et les interactions utilisateur.

2.1 Interface Recruteur — Dashboard Candidatures

L'interface recruteur est organisée autour d'un dashboard central avec KPIs, filtres avancés, et liste triable des candidatures avec scores IA en jauges colorées :

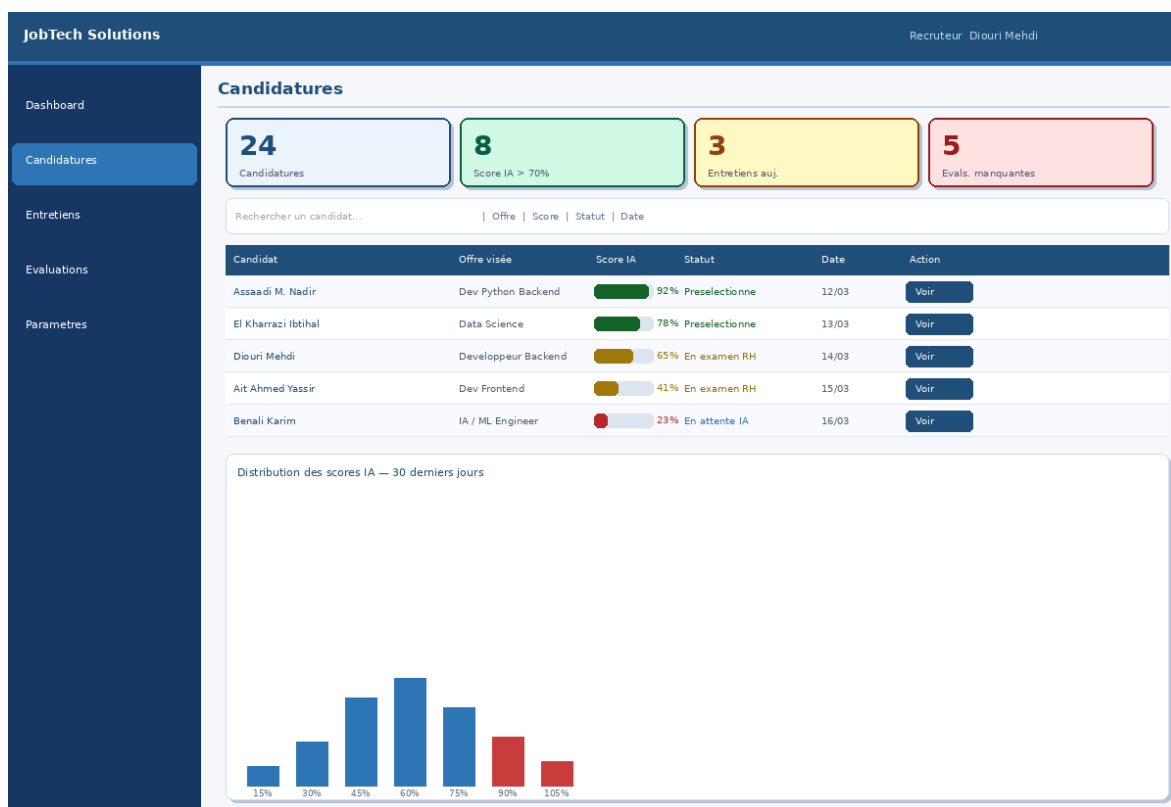


Figure 10 : Maquette — Interface Recruteur (Dashboard Candidatures)

Éléments clés : barre latérale de navigation, 4 cartes KPI (candidatures, scores élevés, entretiens du jour, évaluations manquantes), filtres multicritères, tableau avec jauges de score (vert $\geq 70\%$, orange 40-69%, rouge $< 40\%$), et mini-graphique de distribution des scores.

2.2 Interface Candidat — Suivi des Candidatures

L'interface candidat présente chaque candidature sous forme de carte avec un stepper visuel en 5 étapes montrant la progression dans le processus :

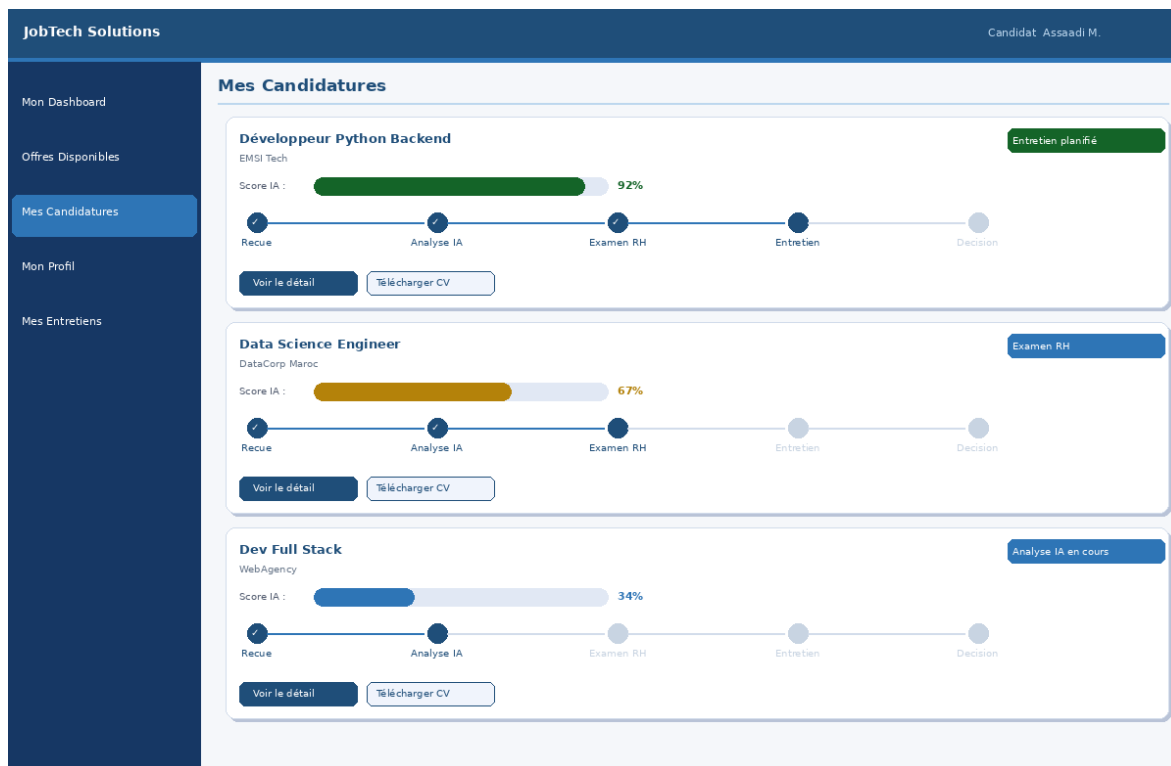


Figure 11 : Maquette — Interface Candidat (Suivi des candidatures)

Chaque carte affiche : titre du poste, entreprise, badge de statut coloré, jauge du score IA, et stepper (Reçue → Analyse IA → Examen RH → Entretien → Décision). Des boutons permettent de voir le détail ou télécharger le CV soumis.

3. Interfaces de l'Application

2.1 Espace Candidat

L'espace candidat est conçu pour une expérience fluide en trois étapes : inscription → dépôt de candidature → suivi. Les interfaces développées comprennent :

- Page d'inscription avec validation en temps réel (format email, force du mot de passe) et confirmation par email via Celery.
- Dashboard candidat : liste des candidatures soumises avec statut d'avancement sous forme de stepper visuel (Reçue → En analyse IA → En traitement RH → Entretien → Décision) et score de compatibilité affiché.
- Formulaire de candidature multi-étapes (1. Choix de l'offre, 2. Informations personnelles, 3. Upload CV) avec barre de progression et indicateur d'analyse IA en temps réel après soumission.
- Espace entretiens : calendrier des rendez-vous planifiés avec détails (date/heure, lieu, lien visio, contacts).

2.2 Espace Recruteur

Optimisé pour la productivité et la prise de décision rapide :

- Dashboard avec alertes intelligentes : candidatures à score IA > 80% non traitées, entretiens du jour, évaluations manquantes.
- Liste des candidatures avec tri multi-critères (score IA décroissant, date, statut) et mode comparaison côte-à-côte de 2 à 3 profils.
- Fiche candidat enrichie : résumé NLP des compétences extraites, radar chart des compétences, score par catégorie (technique/exp./formation), historique des échanges.

- Interface d'entretien avec chronomètre, notes AJAX sauvegardées toutes les 30s, accès rapide à la fiche candidat, et bouton de clôture déclenchant la génération automatique de la fiche d'évaluation.

2.3 Espace Responsable RH

Vision stratégique et opérationnelle sur l'ensemble du processus RH :

- Dashboard analytique : funnel de recrutement (candidatures → entretiens → offres acceptées), délai moyen par étape, coût par embauche, taux de conversion global.
- Gestion des offres avec éditeur rich text pour les descriptions, système de tags pour les compétences (alimentant directement le scoring IA), et prévisualisation avant publication.
- Campagne d'entretiens annuels : vue calendrier trimestrielle, alertes pour les collaborateurs sans entretien planifié, export des convocations en masse.
- Centre de rapports : génération de synthèses PDF agrégées par département, export CSV pour intégration dans des outils tiers (Excel, Power BI).

2.4 Espace Administrateur

Contrôle complet sur le système avec des outils de supervision :

- Gestion des comptes utilisateurs : création en masse (CSV), désactivation temporaire, réinitialisation de mot de passe, modification des rôles avec historique des changements.
- Journal d'audit complet : recherche full-text sur les logs, filtres par utilisateur/action/période, export des logs pour audit de conformité.
- Configuration système : personnalisation des critères d'évaluation par département, gestion des templates d'emails (HTML responsive), seuils d'alerte du scoring IA.
- Tableau de bord technique : charge serveur (CPU/RAM workers Celery), file d'attente des tâches, statistiques d'utilisation par module, alertes de performance.

3. Implémentation du Pipeline IA

3.1 Module NLP — Extraction des Informations de CV

Lorsqu'un candidat soumet son CV, le pipeline NLP s'exécute en 4 phases via un worker Celery :

1. Conversion en texte brut : PyPDF2 pour les fichiers PDF, python-docx pour les DOCX. Nettoyage des caractères spéciaux et normalisation des espaces.
2. Segmentation sémantique : découpage du texte en sections (Expériences, Formations, Compétences, Langues) basé sur des patterns de titres courants en français et anglais.
3. Extraction d'entités (NER) : SpaCy avec le modèle `fr_core_news_lg` pour extraire noms, dates, intitulés de postes. BERT fine-tuné sur un corpus de CV pour extraire les compétences techniques avec une précision > 85%.
4. Normalisation et insertion : les données extraites sont normalisées (dates au format ISO, compétences en minuscules, dédoublonnage) puis insérées dans les tables correspondantes de MySQL.

3.2 Module ML — Calcul du Score de Compatibilité

Le score de compatibilité entre un profil candidat et une offre d'emploi repose sur un algorithme de similarité vectorielle :

5. Vectorisation : les compétences requises (définies par le RH) et les compétences extraites du CV sont transformées en vecteurs numériques TF-IDF enrichis par les embeddings BERT pour capturer la similarité sémantique ("Python" ≈ "Django").

6. Calcul de similarité cosinus : Scikit-learn calcule le cosinus de l'angle entre le vecteur candidat et le vecteur offre. Un angle proche de 0° donne un score proche de 100%.
7. Pondération multi-facteurs : le score brut est pondéré par l'adéquation du niveau de formation (−10% si sous-qualifié), le nombre d'années d'expérience (bonus si au-delà des exigences) et la maîtrise des langues requises.
8. Affichage : le score final (0–100%) est stocké dans `score_ia` et affiché via une jauge colorée :  < 40% (profil inadapté),  40–70% (à considérer),  > 70% (profil recommandé).

★ *Ce chapitre a présenté la réalisation concrète de JobTech Solutions : infrastructure de déploiement robuste, interfaces utilisateur fonctionnelles pour les 4 espaces, et implémentation détaillée du pipeline IA. Le projet est en développement actif avec les modules S1-S3 livrés et les modules IA (S4-S5) en cours.*

Annexe A : Règles de Gestion Métier

Les règles de gestion métier définissent les contraintes fonctionnelles et comportementales que le système doit respecter en toutes circonstances. Ces règles sont indépendantes de l'implémentation technique.

RG-01 à RG-05 : Candidatures

RG-01	Unicité candidature	Un candidat ne peut soumettre qu'une seule candidature par offre d'emploi. Toute tentative de double candidature est rejetée avec un message d'erreur explicite.	Bloquante
RG-02	Format CV	Le CV uploadé doit être au format PDF ou DOCX et ne pas dépasser 5 Mo. Tout autre format ou taille supérieure est rejeté avant traitement.	Bloquante
RG-03	Score IA obligatoire	Aucune candidature ne peut passer au statut 'Examen RH' sans que le pipeline IA ait calculé un score_ia. Si le pipeline échoue, le RH est notifié pour déclencher une relance manuelle.	Bloquante
RG-04	Seuil de score minimal	Toute candidature avec un score_ia < 20% est automatiquement marquée 'Refusée' sans intervention humaine, sauf si le recruteur la réhabilite manuellement.	Paramétrable
RG-05	Délai d'analyse IA	Le pipeline IA doit terminer son exécution dans un délai maximum de 5 minutes après l'upload. Au-delà, une alerte est envoyée à l'administrateur.	Non bloquante

RG-06 à RG-10 : Entretiens et Évaluations

RG-06	Évaluation obligatoire	Tout entretien ne peut être clôturé sans qu'une fiche d'évaluation complète soit soumise. Les 5 critères de notation sont obligatoires.	Bloquante
RG-07	Délai d'évaluation	Le recruteur dispose de 48 heures après la clôture d'un entretien pour soumettre son évaluation. Passé ce délai, une alerte est envoyée au Responsable RH.	Non bloquante
RG-08	Notation des critères	Tous les critères d'évaluation (compétences, communication, motivation, adaptabilité, culture fit) doivent être notés entre 1 (insuffisant) et 5 (excellent). Toute valeur hors plage est rejetée.	Bloquante
RG-09	Anonymat de l'évaluation	Le détail des notes d'évaluation n'est jamais communiqué au candidat. Seule la décision finale (Retenu / À reconsidérer / Non retenu) lui est transmise par email.	Bloquante
RG-10	Validation RH	Toute évaluation doit être validée par le Responsable RH avant de générer la notification finale au candidat. Un statut 'En attente validation' bloque l'envoi.	Bloquante

Annexe B : Dictionnaire de Données

Le dictionnaire de données décrit précisément chaque attribut du modèle de données : nom, type SQL, contraintes, valeurs possibles et description métier.

Table : Utilisateur

Attribut	Type SQL	Contraintes	Description
id	INT	PK, AUTO_INCREMENT	Identifiant unique auto-incrémenté de l'utilisateur.
nom	VARCHAR(100)	NOT NULL	Nom de famille de l'utilisateur, en majuscules.
prenom	VARCHAR(100)	NOT NULL	Prénom de l'utilisateur.
email	VARCHAR(255)	NOT NULL, UNIQUE	Adresse email unique servant d'identifiant de connexion.
role	ENUM	NOT NULL, DEFAULT 'candidat'	Rôle RBAC : 'admin', 'rh', 'recruteur', 'candidat'.
password_hash	VARCHAR(255)	NOT NULL	Hash PBKDF2 du mot de passe. Jamais stocké en clair.
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Indique si le compte est actif. FALSE = désactivé par admin.
date_creation	TIMESTAMP	DEFAULT NOW()	Date et heure de création du compte (UTC).

Table : Candidature

Attribut	Type SQL	Contraintes	Description
id	INT	PK, AUTO_INCREMENT	Identifiant unique de la candidature.
cv_file	VARCHAR(255)	NOT NULL	Chemin relatif vers le fichier CV sur le serveur (ex: uploads/cv_2026_03_john.pdf).
date_postulation	DATETIME	NOT NULL, DEFAULT NOW()	Date et heure de soumission de la candidature.
statut_avancement	ENUM	NOT NULL, DEFAULT 'recue'	Statut : 'recue', 'analyse_ia', 'examen_rh', 'entretien', 'retenu', 'refuse'.
score_ia	FLOAT	DEFAULT 0, CHECK(0<=score_ia<=100)	Score de compatibilité calculé par le module ML, entre 0 et 100.
offre_id	INT	FK → Offre_Emploi(id), NOT NULL	Référence à l'offre d'emploi ciblée.
user_id	INT	FK → Utilisateur(id), NOT NULL	Référence au candidat ayant soumis la candidature.

Table : Evaluation

Attribut	Type SQL	Contraintes	Description
id	INT	PK, AUTO_INCREMENT	Identifiant unique de l'évaluation.
competences_rate	TINYINT	NOT NULL, CHECK(1<=val<=5)	Note compétences techniques : 1 (insuffisant) à 5 (excellent).
communication_rate	TINYINT	NOT NULL, CHECK(1<=val<=5)	Note qualité de communication et d'expression.
motivation_rate	TINYINT	NOT NULL, CHECK(1<=val<=5)	Note motivation et intérêt pour le poste.
notes	TEXT	NULL	Commentaires libres détaillés du recruteur (max 2000 chars).
recommandation	ENUM	NOT NULL	Décision finale : 'retenu', 'a_reconsiderer', 'non_retenu'.
entretien_id	INT	FK → Entretien(id), UNIQUE	Référence à l'entretien évalué. UNIQUE : une seule éval par entretien.

Annexe C : Matrice de Traçabilité

La matrice de traçabilité établit le lien entre les besoins fonctionnels (BF), les cas d'utilisation (UC), les modules de développement et les sprints. Elle garantit que chaque besoin est couvert par au moins un cas d'utilisation et un module d'implémentation.

ID	Besoin Fonctionnel	Cas d'Util.	Module	Sprint	Statut
BF-01	Authentification sécurisée multi-rôles	UC-01, UC-02	Auth & Sécurité	S1	☒ Livré
BF-02	Gestion des offres d'emploi	UC-05	Offres d'Emploi	S2	☒ Livré
BF-03	Soumission et suivi candidature	UC-14, UC-15, UC-16	Candidatures	S3	☒ Livré
BF-04	Analyse automatique des CV (NLP)	UC-14, UC-15	NLP Parser	S4	☐ En cours
BF-05	Scoring de compatibilité (ML)	UC-10	ML Scorer	S5	☐ En cours
BF-06	Planification des entretiens	UC-06, UC-11	Entretiens	S5	☐ En cours
BF-07	Évaluation post-entretien	UC-12	Évaluations & PDF	S6	☐ Planifié
BF-08	Génération rapports PDF	UC-07, UC-08	Évaluations & PDF	S6	☐ Planifié
BF-09	Prise de notes temps réel (AJAX)	UC-11	Notes Temps Réel	S6	☐ Planifié
BF-10	Tableaux de bord et KPIs RH	UC-08	Dashboard RH	S7	☐ Planifié
BF-11	Journal d'audit complet	UC-03	Auth & Sécurité	S1	☒ Livré
BF-12	Configuration du système	UC-04	Auth & Sécurité	S8	☐ Planifié

Annexe D : Plan de Tests

Le plan de tests définit les cas de tests unitaires et d'intégration à implémenter pour valider le bon fonctionnement de chaque module. Les tests sont écrits avec le framework unittest de Django.

Tests Unitaires — Module Authentification

ID Test	Méthode testée	Description	Résultat attendu	Priorité
T-AUTH-01	test_user_creation	Créer un utilisateur avec email, password, rôle valides.	User créé, hash PBKDF2 stocké	●
T-AUTH-02	test_duplicate_email	Tenter de créer deux users avec le même email.	IntegrityError levée	●
T-AUTH-03	test_login_success	Se connecter avec email/password corrects.	Session Django créée, HTTP 200	●
T-AUTH-04	test_login_wrong_password	Se connecter avec un mauvais mot de passe.	HTTP 401, session non créée	●
T-AUTH-05	test_rbac_admin_access	Accéder à une vue admin avec rôle admin.	HTTP 200 retourné	●
T-AUTH-06	test_rbac_forbidden	Accéder à une vue admin avec rôle candidat.	HTTP 403 Forbidden retourné	●

Tests Unitaires — Module IA (NLP & ML)

ID Test	Méthode testée	Description	Résultat attendu	Priorité
T-NLP-01	test_cv_text_extraction	Extraire le texte d'un CV PDF valide.	Texte non vide retourné	●
T-NLP-02	test_entity_extraction	Extraire les compétences d'un CV test.	Liste compétences non vide	●
T-NLP-03	test_score_calculation	Calculer le score entre deux vecteurs de compétences.	Score $\in [0, 100]$	●
T-NLP-04	test_score_perfect_match	Calculer le score avec compétences identiques.	Score = 100.0	●
T-NLP-05	test_score_no_match	Calculer le score avec compétences sans intersection.	Score < 10.0	●

T-NLP-06	test_invalid_file_format	Uploader un fichier .exe comme CV.	ValidationError levée	
----------	--------------------------	------------------------------------	-----------------------	---

Tests d'Intégration

ID Test	Scénario	Étapes	Résultat attendu
T-INT-01	Pipeline complet candidature	1. Candidat s'inscrit → 2. Upload CV → 3. Pipeline IA → 4. Score affiché	score_ia > 0 en BDD, email envoyé
T-INT-02	Flux entretien complet	1. RH planifie → 2. Email envoyé → 3. Recruteur clôture → 4. Éval saisie → 5. PDF généré	PDF disponible, statut = 'Terminé'
T-INT-03	Audit trail	1. Admin crée user → 2. RH publie offre → 3. Vérifier AuditLog	2 entrées AuditLog créées
T-INT-04	RBAC end-to-end	Candidat tente d'accéder à une vue recruteur	HTTP 403, pas d'accès

Annexe E : Glossaire Technique

Ce glossaire définit les termes techniques et métier utilisés dans ce document.

Terme	Définition
ATS (Applicant Tracking System)	Logiciel de gestion des candidatures permettant de suivre le parcours d'un candidat de la réception de son CV jusqu'à l'embauche ou le refus.
BERT	Bidirectional Encoder Representations from Transformers. Modèle de langage pré-entraîné de Google, utilisé ici pour l'extraction sémantique de compétences depuis les CV.
Celery	Framework Python de traitement de tâches asynchrones. Permet d'exécuter le pipeline IA en arrière-plan sans bloquer le serveur web.
Class-Based Views (CBV)	Vues Django implémentées sous forme de classes Python héritant de vues génériques (ListView, CreateView, etc.). Réduit la duplication de code.
Compatibilité Score (score_ia)	Pourcentage de 0 à 100 calculé par le module ML indiquant l'adéquation entre les compétences d'un candidat et les exigences d'une offre d'emploi.
Django MVT	Pattern d'architecture Model-View-Template propre à Django. Variante du MVC où le Template gère la présentation et la View contient la logique métier.
Embeddings	Représentation vectorielle numérique d'un texte dans un espace multidimensionnel, capturant le sens sémantique. Utilisés par BERT pour la similarité de compétences.
Gunicorn	Green Unicorn. Serveur WSGI (Web Server Gateway Interface) Python utilisé en production pour servir l'application Django avec plusieurs workers en parallèle.
Migration Django	Fichier Python versionné décrivant les modifications du schéma de base de données. Permet de faire évoluer le modèle de données de manière contrôlée.
NER (Named Entity Recognition)	Tâche de NLP consistant à identifier et classer automatiquement des entités nommées dans un texte (personnes, dates, organisations, compétences techniques).
NLP (Natural Language Processing)	Traitement Automatique du Langage Naturel. Ensemble de techniques permettant à un ordinateur d'analyser, comprendre et générer du texte en langage humain.
ORM (Object-Relational Mapping)	Couche d'abstraction qui permet d'interagir avec la base de données en utilisant des objets Python plutôt que des requêtes SQL directes.
PBKDF2	Password-Based Key Derivation Function 2. Algorithme de hachage de mots de passe utilisé par Django, résistant aux attaques par force brute et par table arc-en-ciel.
Pipeline IA	Séquence d'étapes de traitement automatique : extraction texte CV → NLP SpaCy/BERT → vectorisation → scoring ML → mise à jour BDD.
RBAC	Role-Based Access Control. Modèle de contrôle d'accès où les permissions sont attribuées aux rôles, et les utilisateurs sont assignés à des rôles.
ReportLab	Bibliothèque Python pour la génération programmatique de documents PDF. Utilisée pour créer les rapports d'évaluation personnalisés.
Similarité Cosinus	Mesure de similarité entre deux vecteurs calculant le cosinus de l'angle qui les sépare. Proche de 1 = très similaires, proche de 0 = très différents.

SpaCy	Bibliothèque Python de NLP industriel. Utilisée pour la reconnaissance d'entités nommées (NER) dans les CV : extraction de noms, dates, intitulés de postes.
Swim Lane	Couloir de nage dans un diagramme d'activité UML. Délimite les activités par acteur ou par composant du système, clarifiant les responsabilités.
TF-IDF	Term Frequency-Inverse Document Frequency. Méthode de vectorisation de texte pondérant l'importance de chaque terme dans un document par rapport à un corpus.
Worker Celery	Processus Python indépendant qui exécute les tâches asynchrones en arrière-plan. Plusieurs workers peuvent s'exécuter en parallèle pour traiter plusieurs CVs simultanément.

Conclusion Générale

Le projet JobTech Solutions, développé dans le cadre de notre cours Python à l'EMSI, nous a permis de concevoir et d'implémenter une solution web complète répondant à une problématique réelle et concrète : la modernisation de la gestion des entretiens professionnels.

À travers les quatre chapitres de ce cahier des charges, nous avons établi un cadre méthodologique rigoureux. Le premier chapitre a posé le contexte et justifié la pertinence du projet face aux limites des outils existants. Le deuxième a fourni une conception technique approfondie : architecture 5 couches, diagrammes UML complets, schéma ERD et allocation détaillée des tâches. Le troisième a analysé les besoins fonctionnels des 6 modules et documenté les choix technologiques. Le quatrième a présenté les résultats de réalisation avec le détail des interfaces et de l'implémentation IA.

Ce projet nous a permis de mettre en pratique un spectre large de compétences Python : développement web full-stack avec Django, conception de bases de données relationnelles, traitement du langage naturel avec SpaCy et BERT, machine learning avec Scikit-learn, traitement asynchrone avec Celery, et génération de documents PDF avec ReportLab. Il illustre la puissance de l'écosystème Python pour construire des applications complètes et intelligentes.

Les perspectives d'évolution incluent l'ajout d'une API REST pour l'interopérabilité, le développement d'une application mobile, l'intégration de LLMs (GPT, Claude) pour des analyses de CV encore plus précises et des suggestions automatiques d'entretiens, ainsi que l'ajout d'une fonctionnalité de visioconférence intégrée.



Ce projet démontre que Python, associé à Django, aux bibliothèques IA et aux bonnes pratiques d'architecture logicielle, permet de construire en équipe réduite une solution professionnelle et évolutive sur un domaine métier complexe.

Références

- [1] Django Software Foundation. Django Documentation v4.2. <https://docs.djangoproject.com/>
- [2] Honnibal, M. & Montani, I. spaCy v3 — Industrial NLP. <https://spacy.io/>
- [3] Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. JMLR, 12, 2011.
- [4] Devlin, J. et al. BERT: Pre-training of Deep Bidirectional Transformers. arXiv:1810.04805, 2018.
- [5] MySQL AB. MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/>
- [6] Celery Project. Celery 5.x — Distributed Task Queue. <https://docs.celeryq.dev/>
- [7] ReportLab Inc. ReportLab PDF Library User Guide. <https://www.reportlab.com/docs/>
- [8] Hunter, J.D. Matplotlib 2D Graphics. Computing in Science & Engineering, 9(3), 2007.
- [9] Wolf, T. et al. HuggingFace Transformers. EMNLP 2020. <https://huggingface.co/docs/>
- [10] Nginx Inc. Nginx Documentation. <https://nginx.org/en/docs/>